

February 23<sup>rd</sup>, 2024

# LED Button Assignment Report

Embedded System 3, LAB 01

*Document Version 0.0*

**Edited by:**

Farros Ramzy (3767353)

### **Abstract**

This technical report describes about the basic use of bit registers in the STM32 board to read an input which should trigger an output. The procedure, context, and knowledge that is written in this document is meant to explain how to setup the pin registers of the STM32-F303RE board to recognize an input and an output.

## List of Figures

|                                                                        |    |
|------------------------------------------------------------------------|----|
| Figure 2. 1. 1 STM32-F303RE Board pinout.....                          | 2  |
| Figure 2. 1. 2 LED & push-button wiring diagram.....                   | 3  |
| Figure 2. 2. 1 Table capture of the GPIO port Enabler.....             | 4  |
| Figure 2. 2. 2 Table capture of the GPIO MODE register.....            | 4  |
| Figure 2. 2. 3 Table capture of the GPIO OTYPE register.....           | 5  |
| Figure 2. 2. 4 Table capture of the GPIO SPEED register. ....          | 5  |
| Figure 2. 4. 1 LED sweep & push-button wiring diagram. ....            | 7  |
| Figure 2. 4. 2 GPIO input setup for experiment.....                    | 7  |
| Figure 2. 4. 3 GPIO output setup for experiment. ....                  | 8  |
| Figure 2. 4. 4 Sweep switch method.....                                | 9  |
| Figure 3. 1. 1 Logic analyzer read: LED & button low. ....             | 10 |
| Figure 3. 1. 2 Logic analyzer read: LED high when trigger is high..... | 11 |
| Figure 3. 1. 3 Logic analyzer read: LED is successfully toggled. ....  | 11 |
| Figure 3. 2. 1 LED's Sweep A loop rotation. ....                       | 12 |
| Figure 3. 2. 2 LED's Sweep B loop rotation. ....                       | 13 |

# Table of Contents

|                                                |    |
|------------------------------------------------|----|
| 1. Introduction .....                          | 1  |
| 1.1. Equipment .....                           | 1  |
| 2. Procedure.....                              | 2  |
| 2.1. Preparation.....                          | 2  |
| 2.2. Setting up Output Pin from the GPIO ..... | 3  |
| 2.3. Setting up Input Pin from the GPIO .....  | 6  |
| 2.4. Experiment .....                          | 6  |
| 3. Results .....                               | 10 |
| 3.1. Button toggle result .....                | 10 |
| 3.2. Experiment result .....                   | 12 |
| 4. Conclusion.....                             | 13 |
| 5. Recommendation.....                         | 13 |
| 6. References .....                            | 14 |

# 1. Introduction

This practicum is a learning method to get to know more about the STM32 board, especially on the bit registers which can be usable for the industrial project and practices in the future.

The requirement of this practicum is to tamper with the STM32 board's bit registers and manipulating those registers to read at least an input and write at least an output as its reading result. This method only allowed to use the CMSIS method of coding (hard coded macros and functions required to be made personally).

Other than that, the assignment is to make sure that the doer of this practicum understands how to use the registers properly for this STM32 board.

There are three things that must be done for this practicum. The first thing is to set an output pin of the STM GPIO, the second one is to toggle the output pin by using an input trigger, and the third one is to combine both methods as creative as possible by doing an extra experiment.

As this experiment is using the button item as the input trigger, it is required to use a de-bouncing method to make sure the STM32 board clearly read the input without any buzzing (up and down value) in it.

Every required equipment is listed in this Equipment section below. And every reference that was being used for this practicum are listed along with the external references in the References section, by the end of this document.

## 1.1. Equipment

The equipment needed to do this research practicum are:

1. A programming platform for the STM32 board (STM32CubeIDE)
2. An STM32-F303RE board
3. At least two LEDs (can add more for the experiment purpose)
4. Two pushbuttons
5. A breadboard
6. A bunch of wires
7. A Logic Analyzer
8. One 220  $\Omega$ ,  $\pm 5\%$  resistor for each LED
9. One 1000  $\Omega$ ,  $\pm 5\%$  resistors for each button

## 2. Procedure

### 2.1. Preparation

To start the assignment, it is required to read the GPIO registers in the reference manual of the STM32-F303RE board which is listed in this References section.

After that, the Nucleo User Manual document must be read and understood to plan a setup input and/or output for the GPIO port of the STM32-F303RE.

From that manual, it can be known that the STM32-F303RE pins which are usable for the input and output are presented as in this Figure 2. 1. 1 below.

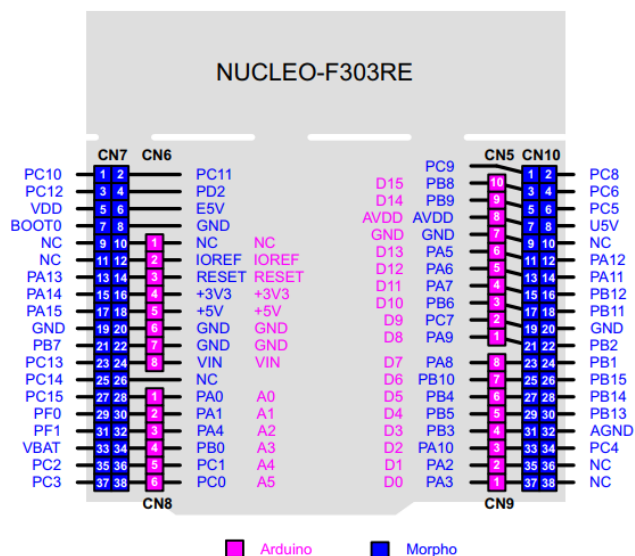


Figure 2. 1. 1 STM32-F303RE Board pinout.

From this Figure 2. 1. 1, it is visible that the pink or purple pins from the board is usable for the Arduino programming.

However, in this practicum, the pins that can be used for the STM GPIO register manipulation are the Morpho pins, reserved for the Nucleo STM32 board.

Nevertheless, there are some pins that cannot be used for external interference such as the internal LED pin (PA5 or PB13) and the internal button (PC13) because these pins are internally built in for the board.

The user of the board can add an external button to control the internal LEDs (PA5 & PB13), but these pins can also still be triggered if the user add an external LED on the same pin.

After deciding the suitable pin and port for the LED and the button, corresponding wire design in this Figure 2. 1. 2 can be drawn.

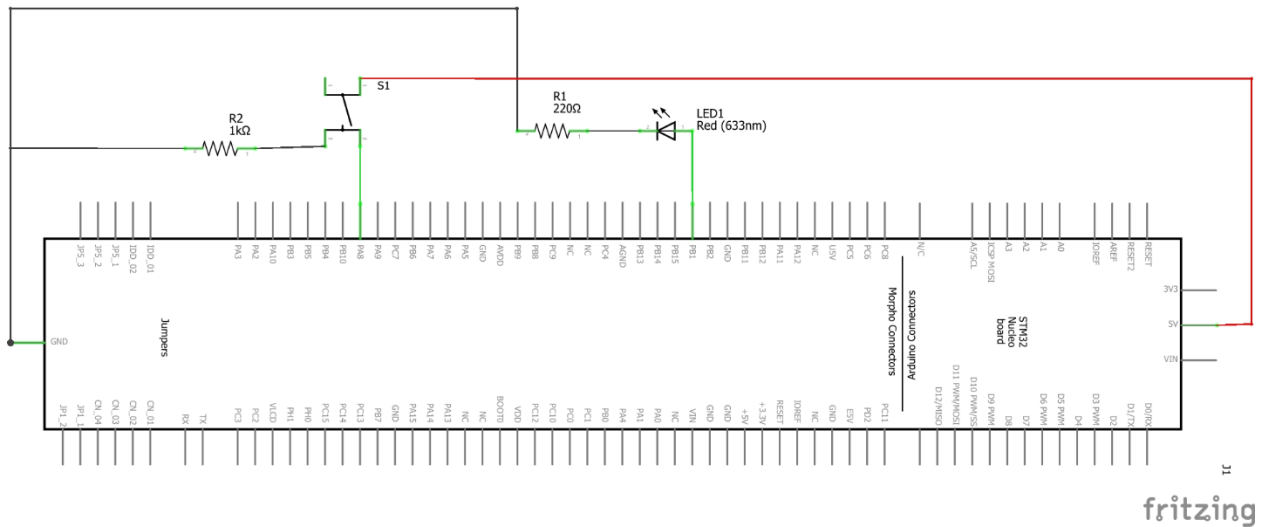


Figure 2. 1. 2 LED & push-button wiring diagram.

From this Figure 2. 1. 2 above, the pin which will be used for the output is PB1 (pin 1 of GPIO B), and the input is on PA8 (pin 8 of GPIO A).

## 2.2. Setting up Output Pin from the GPIO

After understanding the pins of the board, now the setup of one of the bit-register of the STM32F303RE can be programmed. In this part, a pin register for an output will be created.

The idea is to add an LED as the output which is being attached to this pin and make it toggled if any trigger is being received from the input.

To do this, the GPIO port enabler in the reset clock control register (RCC) that controls over the peripheral clock of a certain port of the registers must be applied. The GPIO port enabler have options like enable or disable a certain port of choice in the provided STM device so it can measure which registers that can be used by the software to read or writes the attached pins of the board.

And in this case, the RCC register to activate the peripheral clock which will be used for the LED output is the GPIO port B.

The position of GPIO port B can be seen as GPIOB EN in this Figure 2. 2. 1 below.

|     |     |             |         |     |     |     |       |                           |            |                          |             |            |            |            |                           |
|-----|-----|-------------|---------|-----|-----|-----|-------|---------------------------|------------|--------------------------|-------------|------------|------------|------------|---------------------------|
| 31  | 30  | 29          | 28      | 27  | 26  | 25  | 24    | 23                        | 22         | 21                       | 20          | 19         | 18         | 17         | 16                        |
| Res | Res | ADC34<br>EN | ADC12EN | Res | Res | Res | TSCEN | IOPG<br>EN <sup>(1)</sup> | IOPF<br>EN | IOPE<br>EN               | IOPD<br>EN  | IOPC<br>EN | IOPB<br>EN | IOPA<br>EN | IOPH<br>EN <sup>(1)</sup> |
|     |     | rw          | rw      |     |     |     | rw    | rw                        | rw         | rw                       | rw          | rw         | rw         | rw         | rw                        |
| 15  | 14  | 13          | 12      | 11  | 10  | 9   | 8     | 7                         | 6          | 5                        | 4           | 3          | 2          | 1          | 0                         |
| Res | Res | Res         | Res     | Res | Res | Res | Res   | Res                       | CRC<br>EN  | FMC<br>EN <sup>(1)</sup> | FLITF<br>EN | Res        | SRAM<br>EN | DMA2<br>EN | DMA1<br>EN                |
|     |     |             |         |     |     |     |       |                           | rw         | rw                       | rw          |            | rw         | rw         | rw                        |

1. Only on STM32F303xDxE.

Figure 2. 2. 1 Table capture of the GPIO port Enabler.

After the corresponding GPIO port is enabled, the pin inside that port can be set up manually using some other bit manipulations, depending on the pin number that is going to be used.

The setups are mostly about the mode of the pin (MODER), the speed response of the pin (SPEEDR), and the resistor setup (PUPDR).

This setup can be seen from this Figure 2. 2. 2 below.

To be set in an output mode, the MODER of the chosen pin must be masked to the general-purpose output mode value.

And since the output pin that will be used in the project is PB1, MODER\_1 in GPIO B shall be set to general output purpose mode.

|              |    |              |    |              |    |              |    |              |    |              |    |             |    |             |    |
|--------------|----|--------------|----|--------------|----|--------------|----|--------------|----|--------------|----|-------------|----|-------------|----|
| 31           | 30 | 29           | 28 | 27           | 26 | 25           | 24 | 23           | 22 | 21           | 20 | 19          | 18 | 17          | 16 |
| MODER15[1:0] |    | MODER14[1:0] |    | MODER13[1:0] |    | MODER12[1:0] |    | MODER11[1:0] |    | MODER10[1:0] |    | MODER9[1:0] |    | MODER8[1:0] |    |
| rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw          | rw | rw          | rw |
| 15           | 14 | 13           | 12 | 11           | 10 | 9            | 8  | 7            | 6  | 5            | 4  | 3           | 2  | 1           | 0  |
| MODER7[1:0]  |    | MODER6[1:0]  |    | MODER5[1:0]  |    | MODER4[1:0]  |    | MODER3[1:0]  |    | MODER2[1:0]  |    | MODER1[1:0] |    | MODER0[1:0] |    |
| rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw          | rw | rw          | rw |

Bits 2y+1:2y **MODERy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O mode.

00: Input mode (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

Figure 2. 2. 2 Table capture of the GPIO MODE register.

After setting up the mode register, the type of the output must be determined. In this case, the OTyPER register of PB1 in the board is being set to the Output push-pull mode, which means



writing the register 1 of this Figure 2. 2. 3 below to 0b0 into its reset state, to control its high or low state digitally when toggling the PB1 output later.

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 31   | 30   | 29   | 28   | 27   | 26   | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
| Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. |
|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
| 15   | 14   | 13   | 12   | 11   | 10   | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| OT15 | OT14 | OT13 | OT12 | OT11 | OT10 | OT9  | OT8  | OT7  | OT6  | OT5  | OT4  | OT3  | OT2  | OT1  | OT0  |
| rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   |

Figure 2. 2. 3 Table capture of the GPIO OTYPE register.

After setting up the output type, the speed of the pin can be determined by toggling the OSPEEDR register value in this board. Based on the Figure 2. 2. 4 below, the speed of the LED is set to 0b11 which means the high speed.

|                    |    |                    |    |                    |    |                    |    |                    |    |                    |    |                   |    |                   |    |
|--------------------|----|--------------------|----|--------------------|----|--------------------|----|--------------------|----|--------------------|----|-------------------|----|-------------------|----|
| 31                 | 30 | 29                 | 28 | 27                 | 26 | 25                 | 24 | 23                 | 22 | 21                 | 20 | 19                | 18 | 17                | 16 |
| OSPEEDR15<br>[1:0] |    | OSPEEDR14<br>[1:0] |    | OSPEEDR13<br>[1:0] |    | OSPEEDR12<br>[1:0] |    | OSPEEDR11<br>[1:0] |    | OSPEEDR10<br>[1:0] |    | OSPEEDR9<br>[1:0] |    | OSPEEDR8<br>[1:0] |    |
| rW                 | rW | rW                 | rW | rW                 | rW | rW                 | rW | rW                 | rW | rW                 | rW | rW                | rW | rW                | rW |
| 15                 | 14 | 13                 | 12 | 11                 | 10 | 9                  | 8  | 7                  | 6  | 5                  | 4  | 3                 | 2  | 1                 | 0  |
| OSPEEDR7<br>[1:0]  |    | OSPEEDR6<br>[1:0]  |    | OSPEEDR5<br>[1:0]  |    | OSPEEDR4<br>[1:0]  |    | OSPEEDR3<br>[1:0]  |    | OSPEEDR2<br>[1:0]  |    | OSPEEDR1<br>[1:0] |    | OSPEEDR0<br>[1:0] |    |
| rW                 | rW | rW                 | rW | rW                 | rW | rW                 | rW | rW                 | rW | rW                 | rW | rW                | rW | rW                | rW |

Bits  $2y+1:2y$  **OSPEEDRy[1:0]**: Port x configuration bits ( $y = 0..15$ )

These bits are written by software to configure the I/O output speed.

x0: Low speed

01: Medium speed

11: High speed

*Note: Refer to the device datasheet for the frequency specifications and the power supply and load conditions for each speed.*

Figure 2. 2. 4 Table capture of the GPIO SPEED register.

And the last one is resetting the PUPD\_1 register to 0b00 since the output pin does not use the pull-up/pull-down resistor.

After these setups are finished, the PB1 is now ready to be toggled by the STM system to be HIGH or LOW.

And the next thing that needs to be done is setting up the input trigger to toggle the LED output of PB1.

### 2.3. Setting up Input Pin from the GPIO

Following to the wiring design in Figure 2. 1. 2 above, an input GPIO pin will be set up in this part of the assignment here. The input GPIO is using the GPIO port A. And the input compartment that will be attached to this pin is a press button.

By the design in Figure 2. 1. 2 above, the button input is placed on PA8, which means the pin 8 of GPIO port A. To set the button as an input in PA8, the MODER of the pin must be put into 0b00.

The reason can be viewed in the Figure 2. 2. 2 from the previous procedure above.

After that, the SPEEDR of PA8 can be set up the same as PB1. However, the PUPDR of PA8 must be set to 0b0010 because the requirement is to use the pull-down resistor method as the input of the practicum.

After finishing the setup, the button input can be used in a de-bouncing mode by using the if statements in the main loop program of the system.

Then, the LED's switch can be written by using the BRR (to reset) or BSRR (to set) in a function of the system program using the PA8 as the input and PB1 as the output.

Once the program is running properly, the result must likely be similar to the manual testing result, written in Button toggle result section below this document.

The source code of the corresponding program can be accessed via the git link, written in References section of this document.

### 2.4. Experiment

To make this assignment more fun and interactive, an experiment such as creating a row of sweep LEDs, combined it with the toggling LED-pushbutton assignment can be created.

The concept of this experiment is to switch the sweep mode of the row of the connected LEDs to the STM32 board by using the button.

There are three modes of the sweep LEDs. The first one is the single LED sweep mode, the second one is the multiple LEDs sweep mode, and the last one is the off or reset mode.

To do this, the system needs four more extra LEDs with 220  $\Omega$ ,  $\pm 5\%$  resistors, also a press button with a 1000  $\Omega$ ,  $\pm 5\%$  resistor. The wiring method for this experiment is presented in this Figure 2. 4. 1 below.

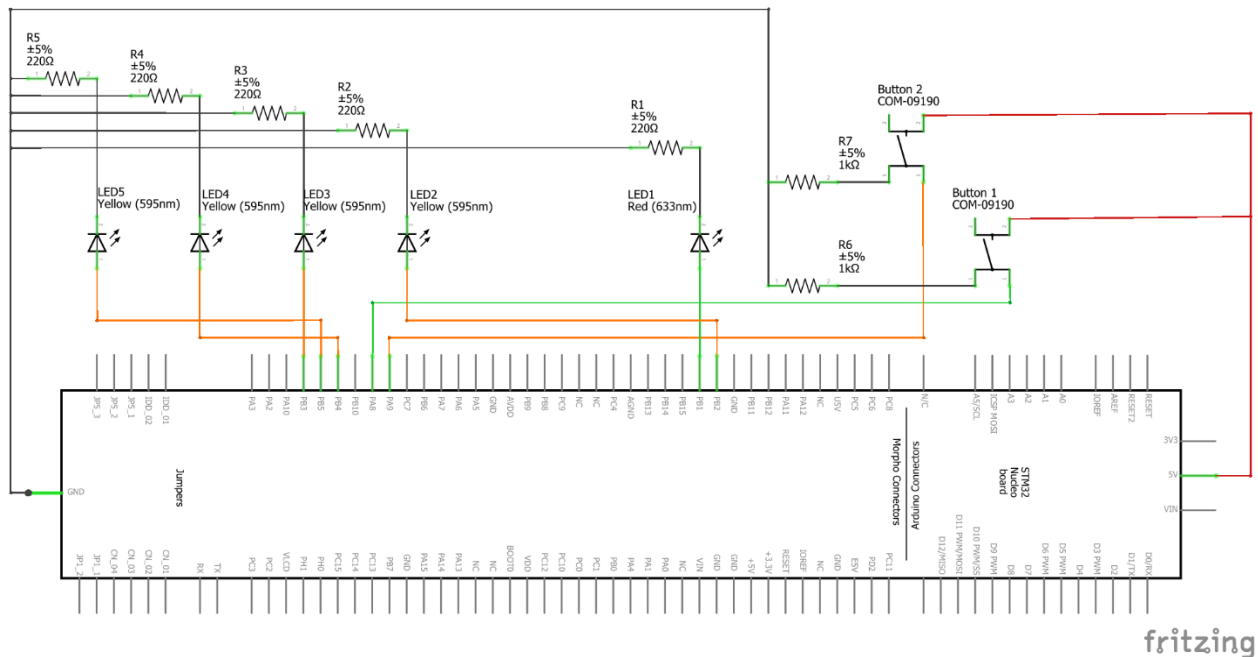


Figure 2. 4. 1 LED sweep & push-button wiring diagram.

To enable the extra LED outputs and an input button, the previous setup can be extended, following the necessary pins needed which is depicted in Figure 2. 4. 1 above.

As for example, by using the “|” (OR) pipe symbol in the GPIO A and the GPIO B setup.

This Figure 2. 4. 2 below is an example of the extended GPIO A setup for the input buttons.

```
void gpio_a_init(void) {
    //resets both button pins' MODE to 00 as an input.
    GPIOA->MODER &= ~((0x3UL << GPIO_MODER_MODER8_Pos)
        | (0x3UL << GPIO_MODER_MODER9_Pos));
    //resets both button pins' SPEED.
    GPIOA->OSPEEDR &= ~((0x3UL << GPIO_MODER_MODER8_Pos)
        | (0x3UL << GPIO_OSPEEDER_OSPEEDR9_Pos));
    //resets both button input PUPD registers.
    GPIOA->PUPDR &= ~((0x3UL << GPIO_PUPDR_PUPDR8_Pos)
        | (0x3UL << GPIO_PUPDR_PUPDR9_Pos));

    //sets both button pins' SPEED to 01 (medium).
    GPIOA->OSPEEDR |= ((0x1UL << GPIO_OSPEEDER_OSPEEDR8_Pos)
        | (0x1UL << GPIO_OSPEEDER_OSPEEDR9_Pos));
    //sets both button input PUPD registers to 01 (pull-down).
    GPIOA->PUPDR |= ((0x2UL << GPIO_PUPDR_PUPDR8_Pos)
        | (0x2UL << GPIO_PUPDR_PUPDR9_Pos));
}
```

Figure 2. 4. 2 GPIO input setup for experiment.

And this Figure 2. 4. 3 below is an example of the extended GPIO B setup for the output LEDs.

```
void gpio_b_init(void) {  
    //resets all the LEDs' pin MODE.  
    GPIOB->MODER &= ~((0x3UL << GPIO_MODER_MODER1_Pos)  
        | (0x3UL << GPIO_MODER_MODER2_Pos)  
        | (0x3UL << GPIO_MODER_MODER3_Pos)  
        | (0x3UL << GPIO_MODER_MODER4_Pos)  
        | (0x3UL << GPIO_MODER_MODER5_Pos));  
    //resets all LEDs' pin OTYPE to 0 as push-pull.  
    GPIOB->OTYPER &= ~(GPIO_OTYPER_OT_1 | GPIO_OTYPER_OT_2  
        | GPIO_OTYPER_OT_3 | GPIO_OTYPER_OT_4 | GPIO_OTYPER_OT_5);  
    //resets all LEDs' pin SPEED.  
    GPIOB->OSPEEDR &= ~((0x3UL << GPIO_OSPEEDER_OSPEEDR1_Pos)  
        | (0x3UL << GPIO_OSPEEDER_OSPEEDR2_Pos)  
        | (0x3UL << GPIO_OSPEEDER_OSPEEDR3_Pos)  
        | (0x3UL << GPIO_OSPEEDER_OSPEEDR4_Pos)  
        | (0x3UL << GPIO_OSPEEDER_OSPEEDR5_Pos));  
  
    //sets the LEDs' pin MODE to 01 as an output.  
    GPIOB->MODER |= ((0x1UL << GPIO_MODER_MODER1_Pos)  
        | (0x1UL << GPIO_MODER_MODER2_Pos)  
        | (0x1UL << GPIO_MODER_MODER3_Pos)  
        | (0x1UL << GPIO_MODER_MODER4_Pos)  
        | (0x1UL << GPIO_MODER_MODER5_Pos));  
    //sets all LEDs' pin SPEED to medium.  
    GPIOB->OSPEEDR |= ((0x1UL << GPIO_OSPEEDER_OSPEEDR1_Pos)  
        | (0x1UL << GPIO_OSPEEDER_OSPEEDR2_Pos)  
        | (0x1UL << GPIO_OSPEEDER_OSPEEDR3_Pos)  
        | (0x1UL << GPIO_OSPEEDER_OSPEEDR4_Pos)  
        | (0x1UL << GPIO_OSPEEDER_OSPEEDR5_Pos));  
    //resets all LEDs' PUPD register.  
    GPIOB->PUPDR &= ~((0x3UL << GPIO_PUPDR_PUPDR1_Pos)  
        | (0x3UL << GPIO_PUPDR_PUPDR2_Pos)  
        | (0x3UL << GPIO_PUPDR_PUPDR3_Pos)  
        | (0x3UL << GPIO_PUPDR_PUPDR4_Pos)  
        | (0x3UL << GPIO_PUPDR_PUPDR5_Pos));  
}
```

Figure 2. 4. 3 GPIO output setup for experiment.

After all the necessary pins are being enabled, now the sweeping program can be written.

The idea of the sweep is to make a row of LEDs to be on and off, queued by their time and position in a loop.

To do the sweep switch, another debouncing method for the second input button must be created.

After that, a switch method to change the sweep states from each press trigger from the button can be added to the program.

This switch method can be seen in Figure 2. 4. 4 as follows.

```
//Run sweeps per state
if (led_sweep_type == 0) {
    all_sweep_off();
}
else if (led_sweep_type == 1) {
    sweep_a(&sweep_a_counter, &sweep_a_to_left);
} else if (led_sweep_type == 2) {
    sweep_b(&sweep_b_counter, &sweep_b_adder, &sweep_b_reverse,
            &sweep_b_to_left);
}
```

Figure 2. 4. 4 Sweep switch method.

As per-function shown in this Figure 2. 4. 4 above, sweep\_a() is the function to executes the single sweep mode, the sweep\_b() is the function to executes the multiple sweep mode, and the all\_sweep\_off() is the function to turn off all of the sweep LEDs.

The result of the manual testing of this program can be seen in the Experiment result section below this document. And the source file can be accessed via a git link in the References section of this document.

### 3. Results

After doing these practices, some simulations and manual testing can be made by using a logic analyzer or serial monitor. In this case, the manual testing used in this practicum is by using a logic analyzer. The logic analyzer device being used here is The Scana, IKA Logic software where the device version is SQ50.

The result of each testing on each LED device product can be seen below here, as follows, in each section of the practicum.

#### 3.1. Button toggle result

At the beginning of the program when it is being ran, both the LED and the button states should be low, as expected, as what is shown in this Figure 3. 1. 1 below.

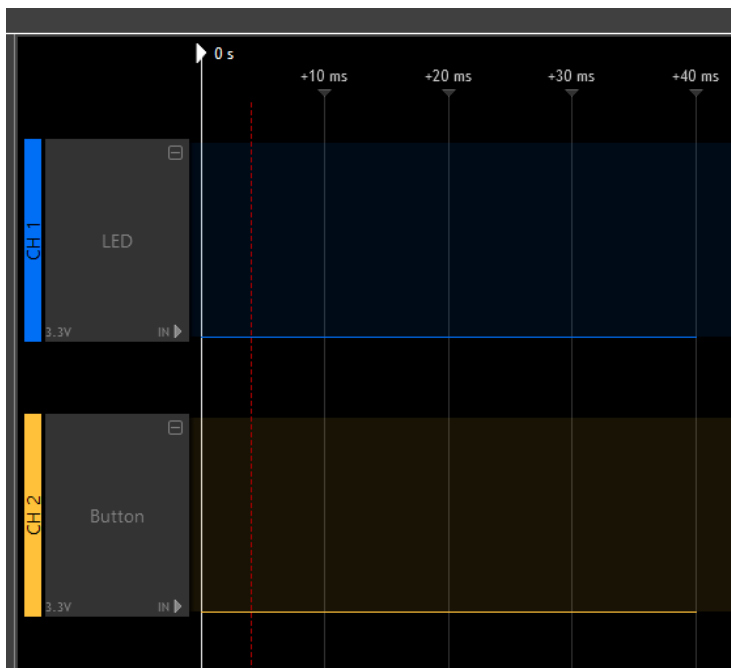


Figure 3. 1. 1 Logic analyzer read: LED & button low.

And when the press button is being pressed, the button is turned to be high, together with the LED which is triggered to be on, as shown in this Figure 3. 1. 2 below.

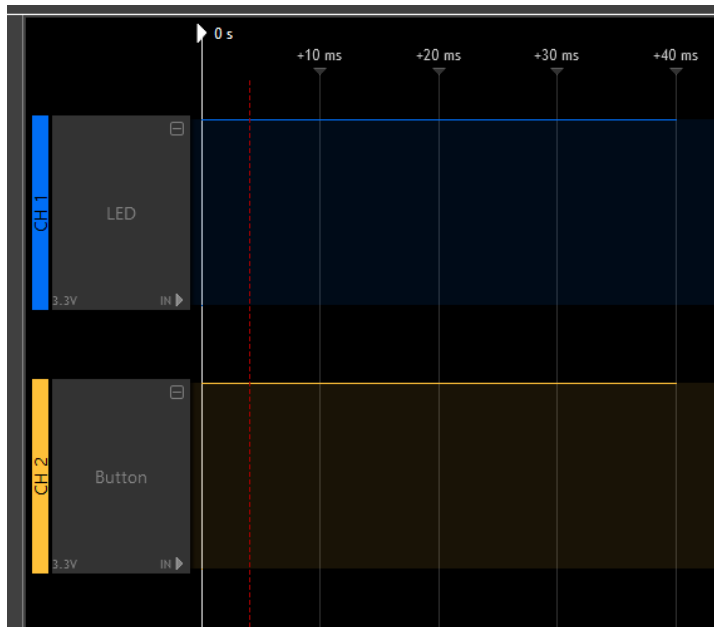


Figure 3. 1. 2 Logic analyzer read: LED high when trigger is high.

And as the button is being released, the LED must be still on high because the logic of the switch LED is only toggling the LED when a press and a release is detected as how it shows in this Figure 3. 1. 3 below.

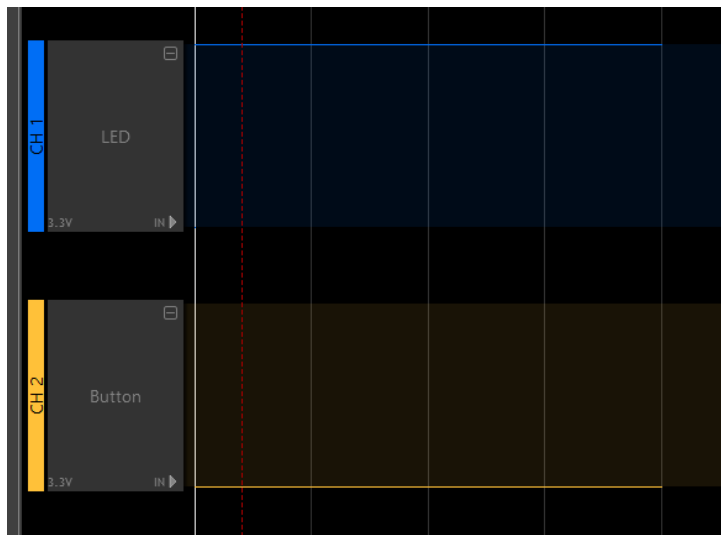


Figure 3. 1. 3 Logic analyzer read: LED is successfully toggled.

### 3.2. Experiment result

The result of the experiment is a little bit different than the previous practice because each LED is switched on and off periodically in a certain queue and timing.

In this Figure 3. 2. 1 below, it is visible that in the sweep a, each LED is on and off by it's order to forward and backward.

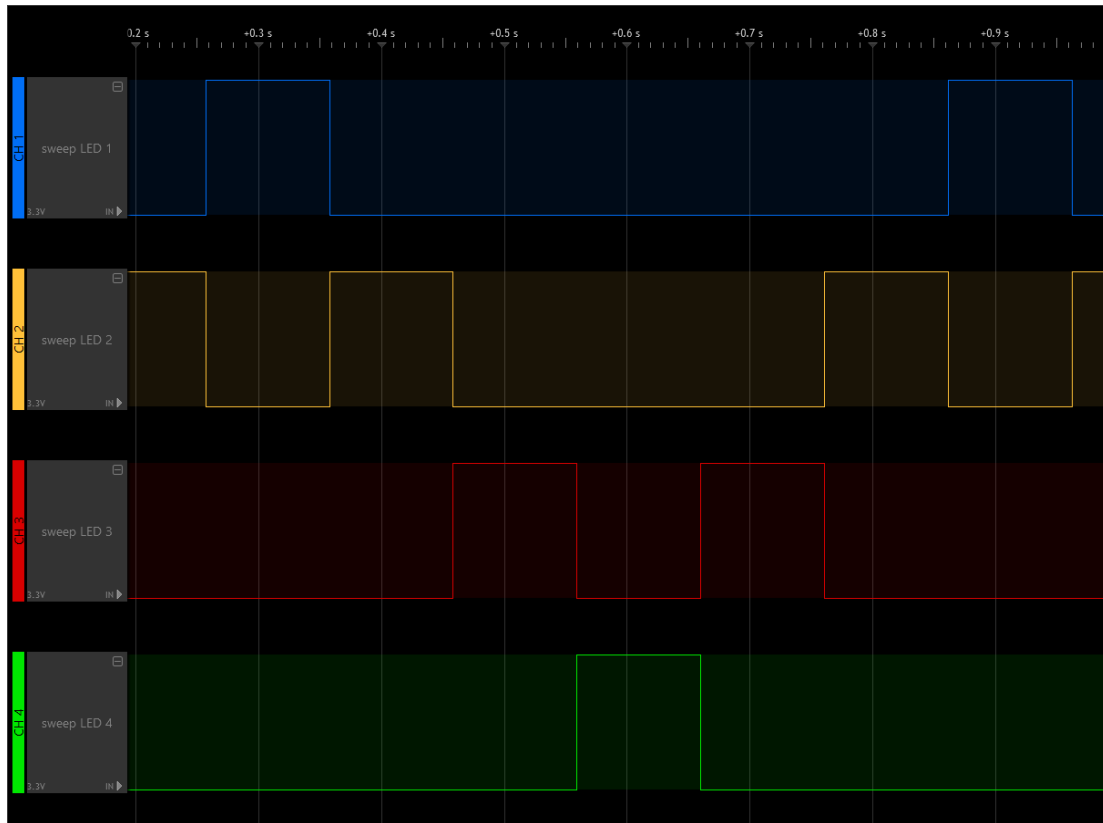


Figure 3. 2. 1 LEDS's Sweep A loop rotation.



And in this Figure 3. 2. 2 below is the simulation of the sweep b, where all LEDs turned on, one by one, by their order, before they go off, one by one, by their order.

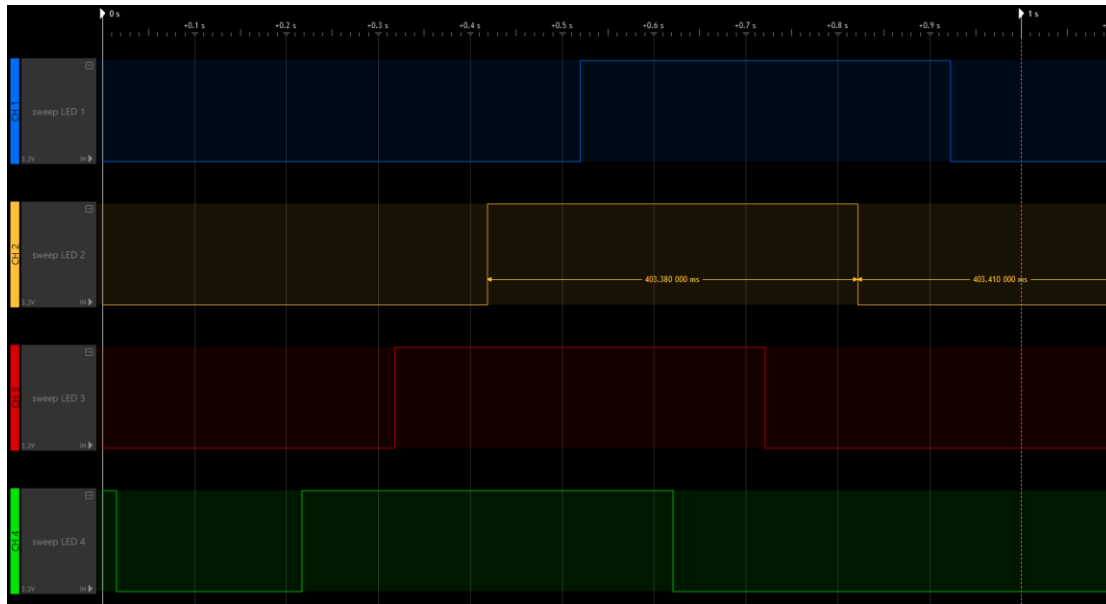


Figure 3. 2. 2 LED's Sweep B loop rotation.

## 4. Conclusion

After doing this practicum, it is becoming so handy to understand the purpose of each register of the STM32 board, and how to use them properly in the CMSIS way of coding.

And based on the practicum, it is known that an output and input can be placed within the same port register as long as the pin manipulation is written correctly, however, it will significantly decrease the number of pins that can be used for external interference, knowing that some pins are also used by the internal items' input and output such as PA5 and PC13.

The internal input and output pins can be interfered by the external input or output, but it will affect both pins at the same time which is not good to be practice with.

## 5. Recommendation

By doing this practicum and its experiment, it seems like an interrupt method can be handy to be used, to make sure that the button switch of the single LED toggling does not affect the other LEDs that is doing the sweeping action. The delay in the de-bouncing method somehow blocked the process of the sweeping action of these LEDs whenever the input button is being used.

## 6. References

Along this practicum, such documents and references are being used.

- DM00043574-ReferenceManual\_STM32F303RE([www.st.com](http://www.st.com)), January 2017
- UM1724 USER manual STM32 Nucleo-64 boards (MB1136) ([www.st.com](http://www.st.com)), April 2019
- Arduino Pull-up Pull-down Resistor ([https://arduinogetstarted.com/fag/arduino-pull-up-pull-down-resistor?utm\\_content=cmp-true](https://arduinogetstarted.com/fag/arduino-pull-up-pull-down-resistor?utm_content=cmp-true))
- Resistor Color Code Calculator (<https://www.digikey.com/en/resources/conversion-calculators/conversion-calculator-resistor-color-code>)

Git links to access the source file of the assignment:

- Basic Push-button LED:  
[https://git.fhict.nl/l424848/t-3-pers-spring-24-farros-ramzy/-/tree/main/es/1-gpio/1\\_LED\\_button\\_a](https://git.fhict.nl/l424848/t-3-pers-spring-24-farros-ramzy/-/tree/main/es/1-gpio/1_LED_button_a)
- Sweep mode experiment:  
[https://git.fhict.nl/l424848/t-3-pers-spring-24-farros-ramzy/-/tree/main/es/1-gpio/1\\_LED\\_button\\_b](https://git.fhict.nl/l424848/t-3-pers-spring-24-farros-ramzy/-/tree/main/es/1-gpio/1_LED_button_b)